Evaluationsdesign

Operatives Arbeitsdokument zur Masterarbeit "Interaktive Kontextkuratierung für Coding Agents". Enthält das Detail, das im Exposé keinen Platz hat: Bedingungen, Metrikdefinitionen, Ablaufprotokolle, Erhebungsinstrumente.

Stand: 2026-08-05 **Bezug:** `expose_main.tex` (v4), Zielsetzung S. 5 **Artefakt:** globaler `codebase-graph`-MCP, Studiengegenstand ist die stepwise-UI (`get_context_stepwise`)

---

1. Rahmenbedingungen

Punkt	Festlegung
Studien-Codebase	TypeScript/JS-Projekt des Betreuers
Probanden	5–10 Studierende aus dem Kurs des Betreuers
Bearbeitungszeit	3–4 Monate
Aufgabenzahl	3 (neu zu erstellen, inkl. Golden Sets)
Parser	`parse.js` parst TS/JS in-process, kein Anpassungsbedarf

Nicht mehr gültig: die Studienanlage B1–B4 aus `graph/docs/study-infrastructure.md` und die Aufgaben `T1-01`…`T3-01`. Beide gehören zum früheren Thema.

---

2. Vorabprüfung des Betreuer-Projekts

Go/No-Go, bevor Aufgaben geschrieben werden.

Nr.	Prüfung	Kriterium	Wie prüfen
V1	Größe	≥ 150 Quelldateien	Dateizählung über `src/`
V2	Graph baut durch	Keine Fehler, plausible Knotenzahl	`parse_codebase` + `import_graph`
V3	Kanten vorhanden	`IMPORTS` und `CALLS` verbinden den Graphen; keine isolierten Inseln	Neo4j: Anteil Knoten mit Grad 0
V4	Beschreibungsgüte	Stichprobe von 20 Knoten fachlich aussagekräftig	Manuelle Sichtung
V5	Retrieval sinnvoll	Drei Testanfragen liefern nachvollziehbare Seeds	`get_context` mit Beispielaufgaben
V6	Aufgabenpotenzial	≥ 3 Stellen mit Abhängigkeiten über 2+ Hops	Manuelle Codesichtung
V7	Freigabe	Schriftliche Zustimmung des Betreuers; geklärt, ob Codeausschnitte in der Thesis erscheinen dürfen	Absprache

Wenn V1 scheitert: Kuratierung hat keinen Spielraum, weil automatisches Retrieval bei kleinem Suchraum fast alles findet. Dann Codebase wechseln oder um Distraktor-Module ergänzen.

Wenn V3 scheitert: Traversierung ist wirkungslos, das gesamte Strukturargument (FF5) verliert seine Grundlage. Vorrangig beheben.

---

3. Forschungsfragen und ihre Zuordnung

	Frage	Vergleich	Stufe
FF1	Gestaltung eines Retrievals, das Auswahlgründe offenlegt und schrittweise modifizierbar macht	Artefaktbeschreibung	—
FF2	Treffsicherheit der Relevanzurteile von Entwicklern	D vs. D′	4
FF3	Verändert Kuratierung die Kontextgüte?	C vs. D, C vs. D′	1, 4
FF4	Wirkt sich das auf Lösungsqualität, Schritte, Tokens aus?	A/B/C/D	3, 4
FF5	Strukturierte vs. flache Übergabe desselben Knotensatzes	B vs. C	3
FF6	Interaktionsaufwand, Nachvollziehbarkeit, Kontrolle, Vertrauen	Nutzerstudie	4

Regel für die Konsistenzprüfung: Jede Zeile braucht einen Vergleich, jeder Vergleich eine Zeile. Bleibt eine Seite leer, fehlt entweder die Frage oder die Messung.

---

4. Bedingungen

	Bezeichnung	Konkrete Umsetzung
A	Baseline, reaktiv	Claude Code mit Read/Grep/Glob, MCP deaktiviert
B	Graph automatisch, flach	`get_context({ task, hopDepth: 2, outputFormat: "summary" })`
C	Graph automatisch, strukturiert	`get_context({ task, hopDepth: 2, outputFormat: "structured" })`
D	Graph kuratiert	`get_context_stepwise({ task })`, Proband steuert Seeds, Traversierungstiefe, Auswahl, Gewichtung
D′	Oracle-Kuratierung	Subgraph aus C, automatisch gegen Golden Set optimiert

Kritische Voraussetzung für B vs. C: Beide Bedingungen müssen denselben Knotensatz liefern und sich ausschließlich im Renderer unterscheiden. Sonst vermischt der Vergleich Retrieval- und Darstellungseffekte und FF5 ist nicht beantwortbar. Das entspricht der offenen Entscheidung D-4 in `analysis/status_2026-07-26.md` und muss vor Stufe 3 fallen.

Zur Kontrolle konstant halten: Modellversion, Temperatur, Seed-Parameter (`semDelta`, `semFloor`, `semTopK`), `hopDepth`, Systemprompt, Projektzustand (`git checkout <baseCommit>`). Alle Werte je Lauf mitloggen.

A, B, C und D′ brauchen keine Probanden. Sie sind skriptbar und beliebig wiederholbar. Nur D braucht Menschen.

---

5. Metrikdefinitionen

5.1 Kontextgüte

Sei `R` die Menge der an den Agenten übergebenen Knoten, `G` der Golden Set der Aufgabe.

```
Precision = |R ∩ G| / |R|
Recall    = |R ∩ G| / |G|
F1        = 2 · P · R / (P + R)
Hit@k     = 1, wenn mindestens ein Knoten aus G unter den k höchstgewichteten in R liegt
```

Granularität getrennt berichten. Datei- und Funktionsknoten nicht vermischen:

- *Dateiebene:* Ein File-Knoten zählt als Treffer, wenn sein Pfad in `goldenFiles` steht.
- *Funktionsebene:* Ein Function-Knoten zählt als Treffer, wenn Name und Datei in `goldenFunctions` stehen.

Eine Funktion, die in einer Golden-Datei liegt, aber nicht selbst im Golden Set steht, ist **kein** Treffer auf Funktionsebene. Diese Regel vorab festlegen und nicht nachträglich ändern.

5.2 Traversierungsprofil

Je Treffer die Hop-Distanz zum nächstgelegenen Seed festhalten (0 = Seed selbst). Anteil der Treffer bei Hop ≥ 1 belegt den Beitrag der Traversierung gegenüber reiner Vektorsuche — das ist der direkte empirische Beleg für das Graph-Argument aus Abschnitt 2.3 des Exposés.

5.3 Kuratierungsgüte (nur Bedingung D)

Bezogen auf den automatisch vorgeschlagenen Subgraphen `R_auto` und die Nutzerauswahl `R_user`:

Größe	Definition
Korrekt entfernt	`
Fälschlich entfernt	`
Korrekt ergänzt	`
Fälschlich ergänzt	`
Δ-Oracle	F1 von `R_user` gegen `R_oracle`
Durchwink-Quote	Anteil der Kuratierungsschritte ohne jede Änderung

Δ-Oracle ist das Kernmaß für FF2: Es sagt, wie nah reale Entwickler an das Erreichbare kommen.

5.4 Agentenmetriken

Metrik	Quelle
Schritte	Anzahl Assistant-Turns im Transcript
Werkzeugaufrufe	gruppiert nach Werkzeug
Input-/Output-Tokens	`usage`-Feld im Transcript
Laufzeit	Zeitstempel erster bis letzter Turn
Akzeptanzquote	erfüllte / gesamte Akzeptanzkriterien

5.5 Aufwandsmetriken (nur D)

Kuratierungsdauer, Anzahl Interaktionen (An-/Abwahl, Reseed, Hop-Änderung, Gewichtung), Anzahl Kuratierungsrunden bis zur Bestätigung.

---

6. Stufe 1 — Oracle-Vorabprüfung

Zweck: Obergrenze dessen bestimmen, was Kuratierung überhaupt erreichen kann. **Aufwand:** ~3 Tage. **Zeitpunkt:** früh, vor der Nutzerstudie.

Ablauf je Aufgabe:

1. Subgraph aus Bedingung C abrufen
2. Vier Varianten bilden:
   - `O_del` — alle Knoten außerhalb `G` entfernt (isolierter Precision-Effekt)
   - `O_add` — fehlende Knoten aus `G` ergänzt (isolierter Recall-Effekt)
   - `O_sort` — Knotensatz unverändert, nach Golden-Zugehörigkeit umsortiert (Gewichtungseffekt)
   - `O_full` — alle drei kombiniert
3. Agent unter jeder Variante laufen lassen, 3 Wiederholungen
4. Gegen Bedingung C vergleichen

Go/No-Go-Regel: Bringt `O_full` gegenüber C keine erkennbare Verbesserung bei Akzeptanzquote oder Schrittzahl, wird die Nutzerstudie mit hoher Wahrscheinlichkeit einen Nulleffekt messen. Dann vor der Erhebung nachsteuern — schwierigere Aufgaben, größerer Suchraum, mehr Distraktoren. Dieser Befund ist auch dann berichtenswert: Er sagt etwas darüber aus, wann Kuratierung sich lohnt und wann nicht.

Die Aufspaltung in `O_del` / `O_add` / `O_sort` ist mehr als Fleißarbeit. Sie beantwortet, *welche* Art von Eingriff wirkt — Wegnehmen, Hinzufügen oder Umsortieren. Falls nur Umsortieren wirkt, ist das ein direkter Beleg für die Positionsabhängigkeit aus Abschnitt 2.4 des Exposés (Liu u. a. 2023, Wu u. a. 2025).

---

7. Stufe 2 — SWE-bench als Retrieval-Benchmark

Zweck: Literaturanschluss und quantitative Absicherung auf realistisch großen Codebases. **Aufwand:** ~1,5 Wochen. **Zeitpunkt:** ab Woche 1, unabhängig von allem anderen.

Ohne Ausführung. Keine Docker-Umgebung, keine Testläufe, keine Agentenläufe, keine LLM-Kosten. Gemessen wird ausschließlich der Abruf.

Ablauf:

1. 3–5 Repositories aus SWE-bench Verified auswählen, je ~15 Instanzen
2. Je Repository: klonen, auf `base_commit` der ersten Instanz auschecken, Graph vollständig bauen
3. Je Instanz: `git checkout <base_commit>`, inkrementellen Import laufen lassen, `get_context({ task: problem_statement })` aufrufen
4. Ground Truth aus dem Gold-Patch ableiten: geänderte Dateien und Funktionen
5. Precision, Recall, F1, Hit@k und Traversierungsprofil berechnen

Baseline: reine Vektorsuche ohne Traversierung, gleiche Seed-Parameter. Der Unterschied isoliert den Beitrag des Graphen.

Warum wenige Repos statt vieler Instanzen: Der vollständige Graphaufbau ist der teure Schritt. Bei Gruppierung nach Repository fällt er 3–5 Mal an statt 45 Mal; zwischen Instanzen desselben Repos reicht der inkrementelle Import.

Einschränkung, die ins Kapitel gehört: Die im Gold-Patch geänderten Dateien sind eine *untere Schranke* für den benötigten Kontext — Code, den man zum Verstehen braucht, aber nicht ändert, fehlt darin. Recall wird dadurch systematisch unterschätzt, und zwar für alle Bedingungen gleichermaßen.

---

8. Stufe 3 — Automatisierte Läufe auf dem Betreuer-Projekt

Umfang: 3 Aufgaben × 4 Bedingungen (A/B/C/D') × 3 Wiederholungen = 36 Läufe. **Aufwand:** ~1,5 Wochen inkl. Skripting. **Durchführung:** über Nacht.

Ablauf je Lauf:

1. `git checkout <baseCommit> -- src/` — Ausgangszustand herstellen
2. Frische Agentensitzung mit dem bedingungsspezifischen Prompt
3. Nach Abschluss: `tsc --noEmit`, ESLint, Aufgabentests ausführen
4. Transcript auswerten, Metriken in CSV schreiben

Voraussetzung: Akzeptanzkriterien müssen maschinell prüfbar sein. Bei 36 Läufen ist manuelle Browserprüfung nicht durchführbar. Beim Aufgabenschreiben von Anfang an mitdenken.

Wiederholungen sind nicht optional. Agentenläufe streuen erheblich. Drei Wiederholungen je Zelle sind das Minimum, um Median und Streuung angeben zu können. Temperatur fixieren, Modellversion und Datum protokollieren.

---

9. Stufe 4 — Nutzerstudie

Design: within-subject, Einzeltermine, ~60 Minuten je Person. **Bedingungen:** C (automatisch, strukturiert) vs. D (kuratiert).

9.1 Balancierung

Jede Person bearbeitet 2 der 3 Aufgaben, je eine unter C und eine unter D.

Person	1. Aufgabe	2. Aufgabe
P1	Ta unter C	Tb unter D
P2	Ta unter D	Tb unter C
P3	Tb unter C	Tc unter D
P4	Tb unter D	Tc unter C
P5	Tc unter C	Ta unter D
P6	Tc unter D	Ta unter C

Jede Aufgabe erscheint zweimal unter C und zweimal unter D; die Startbedingung ist über die Gruppe ausgeglichen. Bei 7–10 Personen das Muster ab P1 fortsetzen.

9.2 Ablaufprotokoll

Min	Abschnitt	Inhalt
0–5	Begrüßung	Einwilligung unterschreiben, Aufzeichnung erklären, Freiwilligkeit betonen
5–10	Vorerfahrung	Fragebogen (Abschnitt 10.1)
10–20	Einführung	Codebase-Überblick, Werkzeug demonstrieren, Tutorialaufgabe unter D
20–35	Aufgabe 1	Think-Aloud, Bildschirmaufzeichnung
35–37	Zwischenerhebung	NASA-TLX + Vertrauensskala zur eben bearbeiteten Bedingung
37–52	Aufgabe 2	analog
52–54	Zwischenerhebung	analog
54–60	Abschluss	SUS, halbstrukturiertes Interview

Die Tutorialaufgabe ist nicht optional. Ohne sie misst die erste echte Aufgabe überwiegend Einarbeitung in die Oberfläche statt Kuratierungsverhalten.

9.3 Erhobene Daten

- Vollständiges Kuratierungsprotokoll je Schritt (Was wurde an-/abgewählt, welche Begründung war sichtbar, wie lange dauerte der Schritt)
- Bildschirmaufzeichnung
- Think-Aloud-Audio, transkribiert
- Fragebögen
- Agenten-Transcript

---

10. Erhebungsinstrumente

10.1 Vorerfahrung

1. Jahre Programmiererfahrung insgesamt

2. Erfahrung mit TypeScript/JavaScript (keine / gering / mittel / hoch)
3. Erfahrung mit der Studien-Codebase (keine / oberflächlich / gut)
4. Bisherige Nutzung von Coding-Agents (nie / gelegentlich / regelmäßig / täglich)
5. Vertrautheit mit Graphdatenbanken oder Wissensgraphen (keine / gering / mittel / hoch)

Punkt 3 ist wichtig: Studierende aus dem Kurs des Betreuers kennen das Projekt möglicherweise bereits. Das ist ein ernsthafter Störfaktor für die Relevanzurteile und muss erhoben werden.

10.2 NASA-TLX (Raw TLX, ungewichtet)

Sechs Skalen, je 0–100 in 5er-Schritten, je Bedingung erhoben: geistige Anforderung, körperliche Anforderung, zeitliche Anforderung, Leistung, Anstrengung, Frustration. Auswertung als Mittelwert der sechs Skalen.

Die ungewichtete Variante genügt und spart je Sitzung mehrere Minuten gegenüber dem paarweisen Gewichtungsverfahren.

10.3 Vertrauensskala

Jian, Bisantz und Drury (2000), zwölf Items, siebenstufig. Sechs Items zu Misstrauen (täuschend, hinterhältig, unzuverlässig, verdächtig, vorsichtig, schädlich), sechs zu Vertrauen (zuverlässig, integer, verlässlich, sicher, vertraut, vertrauenswürdig). Misstrauens-Items invertieren, Gesamtwert als Mittelwert.

10.4 SUS

Standard-Systemusability-Skala, zehn Items, fünfstufig. Nur einmal am Ende zur stepwise-UI.

10.5 Interviewleitfaden

1. Woran haben Sie festgemacht, ob ein Knoten relevant ist?
2. Welche der angezeigten Angaben haben Sie tatsächlich genutzt — Ähnlichkeitswert, Traversierungspfad, Dateiname, Beschreibung? Welche haben Sie ignoriert?
3. Gab es Momente, in denen Sie unsicher waren? Woran lag das?
4. Wo hat Sie die Oberfläche ausgebremst?
5. Hätten Sie dem Agenten ohne Prüfung vertraut? Warum, warum nicht?
6. Wann würde sich dieser Aufwand für Sie im Alltag lohnen, wann nicht?

Frage 2 zielt auf den Kern von FF1: ob die offengelegten Begründungen tatsächlich genutzt werden oder nur Beiwerk sind.

---

11. Einwilligung und Datenschutz

Die Teilnehmenden sind Studierende des Betreuers. Daraus folgt ein Abhängigkeitsverhältnis, das behandelt werden muss:

- **Freiwilligkeit** schriftlich zusichern, ausdrücklich keine Kopplung an Note oder Kursleistung, Abbruch jederzeit ohne Angabe von Gründen und ohne Nachteil
- **Rekrutierung möglichst nicht durch den Betreuer selbst**, um wahrgenommenen Druck zu vermeiden
- **Einwilligungserklärung nach DSGVO** vor Beginn, gesonderte Zustimmung zur Bildschirmaufzeichnung

- **Pseudonymisierung**: Teilnehmer-IDs `P01`…`P10`, Zuordnungsliste getrennt von den Forschungsdaten und nach Abschluss der Arbeit gelöscht
- **Aufbewahrung**: Rohdaten lokal, keine Cloud-Synchronisierung, Löschfrist benennen
- **Auskunfts- und Löschrecht** in der Einwilligung nennen

Datenablage

```
studie/
  rohdaten/
    P01/
      transcript-aufgabe1.jsonl
      transcript-aufgabe2.jsonl
      kuratierung.jsonl
      aufzeichnung.mp4
      fragebogen.json
  aufbereitet/
    metriken.csv
    interviews-transkript/
  auswertung/
```

`rohdaten/` und die Zuordnungsliste gehören nicht in die Versionsverwaltung.

---

12. Auswertungsplan

Stufe	Verfahren
1 (Oracle)	Deskriptiv je Variante, gepaart je Aufgabe
2 (SWE-bench)	Precision/Recall/F1 je Instanz, Bootstrap-Konfidenzintervalle, gepaarter Vergleich Graph vs. Vektorsuche
3 (automatisiert)	Median und IQR je Zelle, gepaarte Vergleiche je Aufgabe über die Wiederholungen
4 (Nutzerstudie)	Deskriptiv plus qualitativ; Wilcoxon-Vorzeichen-Rang-Test für C vs. D nur ergänzend

Zur Statistik bei N=5–10 offen sein. Ein Wilcoxon-Test über sechs Paare hat kaum Trennschärfe; exakte p-Werte und Effektstärke `r = Z / √N` angeben und die Aussage nicht darauf stützen. Der quantitative Beleg kommt aus den Stufen 1–3, die beliebig wiederholbar sind. Die Nutzerstudie liefert das Verhalten und die Begründungen, nicht die statistische Absicherung.

Interviews thematisch codieren, Codesystem im Anhang dokumentieren. Bei kleinem N ist die qualitative Auswertung der ergiebigere Teil.

---

13. Zeitplan

Woche	Inhalt
1	Betreuer-Projekt prüfen (Abschnitt 2), Graph bauen, D-4 entscheiden
1–3	*parallel:* Stufe 2 (SWE-bench)
2–3	Artefakt-Härtung (F-14, F-20, F-21), Logging umstellen, Einwilligungsunterlagen
3–5	3 Aufgaben + Golden Sets + automatisierte Akzeptanzchecks
5	Stufe 1 (Oracle) — Go/No-Go
6–7	Stufe 3 (automatisierte Läufe)
8	Pilot mit 1–2 Personen
9–10	Stufe 4 (Nutzerstudie)
11–12	Auswertung
13–15	Schreiben, Puffer

Schreiben läuft ab Woche 1 durchgehend parallel.

---

14. Risiken und Abbruchkriterien

Risiko	Frühwarnzeichen	Reaktion
Codebase zu klein oder zu einfach	V1/V6 in Abschnitt 2 scheitern	Codebase wechseln oder Distraktoren ergänzen — **vor** dem Aufgabenschreiben
Kuratierung bringt nichts	Stufe 1 zeigt keinen Effekt von `O_full`	Aufgaben verschärfen; falls das nicht greift, Befund als Ergebnis berichten und Schwerpunkt auf FF5/FF6 verlagern
Zu wenige Probanden	Weniger als 4 Zusagen bis Woche 8	Auf Fallstudie mit 2–3 Personen reduzieren; Stufen 1–3 tragen die Arbeit
Latenz stört die Sitzungen	Pilot zeigt spürbare Wartezeiten	F-20/F-21 vorziehen
Zeitplan reißt	Stufe 3 nicht bis Ende Woche 7 fertig	Aufgabenzahl von 3 auf 2 senken, nicht die Wiederholungen

Harte Grenze: Ab Woche 11 keine neue Datenerhebung.

---

15. Bewusst nicht Teil der Arbeit

- SWE-bench-Lösungsrate (Docker, Testausführung, Agentenläufe) — nur Retrieval-Metriken
- ContextBench als eigener Datensatz — nur zitiert
- `review-ui` als eigene Bedingung — nur stepwise-ui ist Studiengegenstand
- Sprachen jenseits von TS/JS und Python
- Inferenzstatistik als tragende Aussage der Nutzerstudie